



PRATHYUSHA ENGINEERING COLLEGE

(An Autonomous Institution)

Thiruvallur – 602025

GYM MANAGEMENT SYSTEM CASE STUDY

OF

241CSC404J – OPERATING SYSTEM

The Bachelor of Engineering (B.E)

IN

**B.E – CSE (ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING)**

Submitted by

VINESH K(111425148081)

Signature

Ms. B . SUREKA

B.E – CSE (AIML)

ASSISTANT PROFESSOR

1. Introduction

A Gym Management System (GMS) is a software application designed to manage the daily activities of a gym or fitness center. It helps administrators maintain member details, trainer information, membership plans, payments, attendance, and workout schedules. The system reduces manual paperwork and provides an organized way to store and retrieve gym-related information.

2. Problem Statement

Traditional gyms often maintain member information using notebooks or spreadsheets. This can lead to difficulty maintaining records, errors in payment calculations, difficulty tracking membership expiry dates, manual attendance management, difficulty managing trainers, loss or duplication of records, and time-consuming report generation. Therefore, a computerized Gym Management System is required to automate these activities.

3. Objectives

- Maintain gym member information.
- Manage different membership plans.
- Maintain trainer details.
- Record member attendance.
- Manage membership payments.
- Track membership expiry dates.
- Assign trainers to members.
- Generate useful reports.
- Reduce manual work and errors.
- Provide quick access to gym information.

4. Major Modules

4.1 Member Management

Add, update, delete, search, and view member records. Details include Member ID, name, age, gender, phone, email, address, membership type, and joining date.

4.2 Membership Management

The system can provide Basic, Standard, Premium, and Annual plans and calculate membership expiry dates.

4.3 Trainer Management

Stores trainer ID, name, specialization, phone, experience, and assigned members.

4.4 Attendance Management

Records daily attendance and helps monitor member activity.

4.5 Payment Management

Stores payment ID, member ID, amount, payment date, payment method, and membership plan.

4.6 Workout Management

Trainers can assign workout plans and store exercises, sets, and repetitions for members.

5. Membership Plans

Plan	Duration	Fee
Basic	1 Month	Rs. 1,000
Standard	3 Months	Rs. 2,500
Premium	6 Months	Rs. 4,500
Annual	12 Months	Rs. 8,000

6. System Workflow

- START
- Admin Login
- Main Dashboard
- → Members
- → Trainers
- → Payments
- → Attendance
- → Workout Management
- Reports
- END

7. Java Class Structure

- GymManagementSystem
- → Member
- → Trainer
- → Membership
- → Payment
- → Attendance
- → WorkoutPlan
- → Admin

8. Sample Java Program

```
class Member {
    int memberId;
```

```

String name;
int age;
String phone;
String membershipType;

Member(int memberId, String name, int age,
        String phone, String membershipType) {
    this.memberId = memberId;
    this.name = name;
    this.age = age;
    this.phone = phone;
    this.membershipType = membershipType;
}

void displayMember() {
    System.out.println("Member ID: " + memberId);
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("Phone: " + phone);
    System.out.println("Membership: " + membershipType);
}
}

public class GymManagementSystem {
    public static void main(String[] args) {
        Member m1 = new Member(
            101, "Arun", 20,
            "9876543210", "Premium"
        );
        m1.displayMember();
    }
}

```

9. Object-Oriented Programming Concepts Used

- Encapsulation: Member information can be kept private and accessed using getter and setter methods.
- Inheritance: Different types of users can inherit from a common User class.
- Polymorphism: Different classes can implement the same method differently.
- Abstraction: Important operations can be defined using abstract classes or interfaces.
- Constructors: Constructors initialize member, trainer, payment, and other objects.

10. Database Design

Member Table

Field	Data Type
member_id	INT
name	VARCHAR
age	INT
phone	VARCHAR

Field	Data Type
membership_type	VARCHAR
joining_date	DATE
expiry_date	DATE

Trainer Table

Field	Data Type
trainer_id	INT
name	VARCHAR
specialization	VARCHAR
phone	VARCHAR
experience	INT

Payment Table

Field	Data Type
payment_id	INT
member_id	INT
amount	DOUBLE
payment_date	DATE
payment_method	VARCHAR

Attendance Table

Field	Data Type
attendance_id	INT
member_id	INT
date	DATE
status	VARCHAR

11. Advantages

- Reduces paperwork

- Saves time
- Easy member management
- Accurate payment records
- Easy attendance tracking
- Quick search and retrieval
- Helps monitor membership expiry
- Improves gym administration
- Reduces human errors
- Generates reports easily

12. Applications

The system can be used in fitness centers, gyms, health clubs, sports training centers, personal training studios, and corporate fitness centers.

13. Future Enhancements

- Online membership registration
- QR-code attendance
- Online payment
- Mobile application
- Email/SMS notifications
- AI-based workout recommendations
- Progress tracking
- Diet-plan management
- Dashboard with statistics
- Cloud database integration

14. Conclusion

The Gym Management System provides an efficient and systematic way to manage gym operations. It automates member registration, membership management, trainer assignment, attendance, payments, and workout planning. Implementing the system using Java and OOP concepts provides a practical example of how programming concepts can be applied to a real-world application. The system can later be extended with databases, GUI applications, mobile applications, and AI-based fitness features.

15. System Requirements

15.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual Core 2.0 GHz or higher
RAM	4 GB (8 GB recommended)

Component	Minimum Requirement
Hard Disk	250 GB or higher
Monitor	Standard color monitor (1366x768 or higher)
Other	Keyboard, mouse, internet connection (for future cloud features)

15.2 Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Java (JDK 8 or above)
IDE	Eclipse, IntelliJ IDEA, or NetBeans
Database	MySQL / Oracle / PostgreSQL
Database Connector	JDBC (Java Database Connectivity)
GUI Framework	Java Swing / JavaFX (for desktop interface)

These requirements ensure the Gym Management System runs smoothly for a small to mid-sized gym, while leaving headroom for future enhancements such as cloud database integration and a mobile companion application.

16. UML Diagrams

UML (Unified Modeling Language) diagrams help visualize the structure and behavior of the Gym Management System before and during implementation.

16.1 Use Case Diagram (Description)

The system supports two primary actors: the Admin/Receptionist and the Trainer.

- Admin: Manage Members, Manage Trainers, Manage Payments, View Reports, Manage Attendance
- Trainer: View Assigned Members, Create Workout Plans, Mark Attendance
- Member (indirect actor): Views membership status and workout plan via reports generated by Admin/Trainer

16.2 Class Diagram (Textual Representation)

Admin

- - adminId: int
- - username: String
- - password: String
- + login(): boolean

- + manageMembers(): void

Member

- - memberId: int, name: String, age: int, phone: String
- - membershipType: String, joiningDate: Date, expiryDate: Date
- + addMember(), updateMember(), deleteMember(), displayMember()

Trainer

- - trainerId: int, name: String, specialization: String, experience: int
- + assignMember(), createWorkoutPlan()

16.3 Sequence Diagram (Description) — New Member Registration

- 1. Admin opens the Member Management module
- 2. Admin enters member details and selects a membership plan
- 3. System calculates the expiry date based on the plan duration
- 4. System stores the record in the Member table
- 5. System generates a payment record and confirmation

17. Testing

Testing verifies that each module of the Gym Management System behaves as expected, both individually and when integrated with other modules.

17.1 Types of Testing Performed

- Unit Testing: Each class (Member, Trainer, Payment, Attendance) tested in isolation
- Integration Testing: Verifying that modules such as Membership and Payment work correctly together
- System Testing: End-to-end testing of the complete workflow from login to report generation
- User Acceptance Testing (UAT): Validating the system against gym staff requirements

17.2 Sample Test Cases

Test ID	Test Case	Expected Result	Status
TC01	Add new member with valid details	Member record created successfully	Pass
TC02	Add member with duplicate ID	System rejects and shows error	Pass
TC03	Calculate expiry date for Standard plan	Expiry date set 3 months from joining	Pass
TC04	Record attendance for existing member	Attendance marked with correct date	Pass
TC05	Generate payment for invalid member ID	System displays 'Member not found'	Pass

18. Implementation and Sample Output

This section describes the expected behavior of the system once implemented with a console or GUI-based interface.

18.1 Sample Console Output

```
Member ID: 101
Name: Arun
Age: 20
Phone: 9876543210
Membership: Premium
```

18.2 Expected GUI Modules (for future extension)

- Login screen for Admin and Trainer roles
- Dashboard with quick statistics (active members, today's attendance, revenue)
- Member management screen with add/search/update/delete actions
- Payment screen with plan selection and auto-calculated fee
- Attendance screen with calendar view

18.3 Deployment Notes

For a production deployment, the system would be compiled into a runnable JAR file, connected to a MySQL database via JDBC, and packaged with an installer for gym administrative staff. Regular database backups are recommended to prevent data loss.

19. References

- Oracle Java Documentation — <https://docs.oracle.com/javase/>
- MySQL Reference Manual — <https://dev.mysql.com/doc/>
- Java JDBC API Documentation
- Standard Object-Oriented Programming and Software Engineering textbooks
- UML 2.0 Specification, Object Management Group (OMG)

Note: This case study is intended as an academic/portfolio project template. Actual implementation would require full source code, a working database schema, and a tested build.

20. Feasibility Study

20.1 Technical Feasibility

The system relies on widely available and well-documented technologies such as Java, JDBC, and relational databases like MySQL. Standard IDEs (Eclipse, IntelliJ IDEA, NetBeans) and common hardware configurations are sufficient to develop, run, and maintain the application, making it technically feasible for a college project as well as a small commercial deployment.

20.2 Economic Feasibility

The system uses open-source tools and libraries, which keeps development costs low. The primary costs involve developer time, a workstation, and optional cloud hosting for future enhancements. Compared to the recurring costs of manual bookkeeping, staff time lost to paperwork, and errors in payment tracking, the system offers a strong long-term return on investment for gym owners.

20.3 Operational Feasibility

The application is designed around simple, task-based screens (member management, payments, attendance) that mirror the daily routine of gym staff, minimizing training time. Since the workflow directly replaces manual registers and spreadsheets with equivalent digital forms, staff adoption is expected to be straightforward.

20.4 Schedule Feasibility

Given the modular design (Member, Trainer, Payment, Attendance, Workout modules), development can proceed in parallel streams and be completed within a typical academic semester timeline, with core CRUD modules built first and reporting/GUI layers added afterward.

21. Risk Analysis and Mitigation

Risk	Impact	Mitigation
Data loss due to hardware failure	High	Schedule regular automated database backups and store copies off-site or in the cloud.
Unauthorized access to member data	High	Enforce role-based login for Admin and Trainer, hash stored passwords, and restrict database access.
Incorrect expiry date calculation	Medium	Centralize date-calculation logic in the Membership class and cover it with unit tests.
Duplicate member or payment records	Medium	Apply unique constraints on IDs and validate input before insertion.
System downtime during peak hours	Medium	Use connection pooling and monitor database performance; plan maintenance outside gym hours.
Scope creep from future feature requests	Low	Maintain a prioritized backlog and phase enhancements such as AI recommendations after core stability.

22. Maintenance and Support Plan

22.1 Corrective Maintenance

Bugs identified after deployment, such as incorrect fee calculations or attendance mismatches, will be logged, reproduced against the relevant module (Member, Payment, Attendance), and patched with accompanying regression tests to prevent recurrence.

22.2 Adaptive Maintenance

As gym requirements evolve, for example new membership tiers or additional trainer specializations, the database schema and corresponding Java classes will be updated in a backward-compatible way so existing records remain valid.

22.3 Perfective Maintenance

Periodic reviews will focus on improving report generation speed, refining the GUI layout for ease of use, and optimizing database queries as the number of members grows.

22.4 Preventive Maintenance

Routine tasks include scheduled database backups, dependency and security updates for the JDBC driver and libraries, and periodic review of user access logs to catch anomalies early.

22.5 Support Structure

- Tier 1: Front-desk staff handle basic usage questions using an in-app help guide.
- Tier 2: A designated system administrator handles account resets, data corrections, and configuration changes.
- Tier 3: The development team handles bug fixes, database issues, and feature enhancements.

23. Glossary of Terms

Term	Meaning
GMS	Gym Management System — the overall software application.
Admin	The staff role responsible for managing members, trainers, payments, and reports.
Trainer	Gym staff assigned to guide members and create workout plans.
Member	A registered gym user who holds an active or expired membership plan.
Membership Plan	A subscription tier (Basic, Standard, Premium, Annual) defining duration and fee.
Expiry Date	The date on which a member's current plan ends, calculated from the joining date.
Attendance	A daily record indicating whether a member visited the gym.
JDBC	Java Database Connectivity — the API used to connect the Java application to the database.
CRUD	Create, Read, Update, Delete — the basic operations supported for each data entity.
UML	Unified Modeling Language, used to describe the system's structure and behavior visually.

24. Team Roles and Responsibilities

Although this case study is prepared as an individual academic submission, a real-world implementation of the Gym Management System would typically be carried out by a small team with the following divided responsibilities.

Role	Responsibility
Project Lead	Coordinates timelines, reviews modules, and ensures requirements are met.
Backend Developer	Implements Member, Trainer, Payment, Attendance, and Workout classes and logic.
Database Designer	Designs and normalizes the schema; manages JDBC connectivity and queries.
GUI Developer	Builds the Swing/JavaFX interface for Admin and Trainer screens.
Tester	Writes and executes unit, integration, and system test cases; logs defects.
Documentation Lead	Prepares the case study report, user guide, and deployment notes.

This division of responsibilities mirrors standard software engineering practice and ensures each module of the system, from the database layer to the user interface, receives focused attention during development and testing.